How to provision the benchmark environment

Intro

This document will explain how is organized the benchmark environment and how to provision it. Also, we explain how it is being used in our use case for testing a switchover and promotion.

The benchmark environment consists of the following layers:

Database layer

- Source cluster: 3 nodes with a Patroni cluster.

  Hosts:

  - pg12ute-patroni-source-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal
  - pg12ute-patroni-source-02-db-db-benchmarking.c.gitlab-db-benchmarking.internal
  - pg12ute-patroni-source-03-db-db-benchmarking.c.gitlab-db-benchmarking.internal

- Target cluster: 3 nodes with a secondary Patroni cluster, in cascade replication from the source cluster.

  Hosts:

  - pg12ute-patroni-target-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal
  - pg12ute-patroni-target-02-db-db-benchmarking.c.gitlab-db-benchmarking.internal
  - pg12ute-patroni-target-03-db-db-benchmarking.c.gitlab-db-benchmarking.internal

PGBouncer layer

- Pgbouncer webapi cluster: 3 nodes from pgbouncer pointing to the source cluster.

  Hosts:

  - pgbouncer-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal
  - pgbouncer-02-db-db-benchmarking.c.gitlab-db-benchmarking.internal
  - pgbouncer-03-db-db-benchmarking.c.gitlab-db-benchmarking.internal

- Pgbouncer sidekiq cluster: 3 nodes from pgbouncer pointing to the source cluster.

  Hosts:

  - pgbouncer-sidekiq-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal
  - pgbouncer-sidekiq-02-db-db-benchmarking.c.gitlab-db-benchmarking.internal
  - pgbouncer-sidekiq-03-db-db-benchmarking.c.gitlab-db-benchmarking.internal

- Pgbouncer CI webapi cluster: 3 nodes from pgbouncer pointing to the source cluster.

  Hosts:

  - ci-pgbouncer-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal
  - ci-pgbouncer-02-db-db-benchmarking.c.gitlab-db-benchmarking.internal
  - ci-pgbouncer-03-db-db-benchmarking.c.gitlab-db-benchmarking.internal

- Pgbouncer CI sidekiq cluster: 3 nodes from pgbouncer pointing to the source cluster.

  Hosts:

  - ci-pgbouncer-sidekiq-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal
  - ci-pgbouncer-sidekiq-02-db-db-benchmarking.c.gitlab-db-benchmarking.internal
  - ci-pgbouncer-sidekiq-03-db-db-benchmarking.c.gitlab-db-benchmarking.internal

Jmeter layer

- Jmeter: 2 instances to run Jmeter against the pgbouncers that will connect to the database.

  Hosts:

  - jmeter-01-inf-db-benchmarking.c.gitlab-db-benchmarking.internal
  - jmeter-02-inf-db-benchmarking.c.gitlab-db-benchmarking.internal

Console layer

- Console: one console box where we execute the ansible code.

  Host:

  - console-01-sv-db-benchmarking.c.gitlab-db-benchmarking.internal

Monitoring layer

- PGWatch2: one console box where we have the core of the pgwatch2 running.

  Host:

  - pgwatch-01-sv-db-benchmarking.c.gitlab-db-benchmarking.internal

How to initialize the benchmark environment in Terraform

To generate the environment, please consider the following files on the terraform repo:

- environments/db-benchmarking/pgute12.tf
- environments/db-benchmarking/main.tf

Execute a `tf apply` in the folder from the benchmark environment:

`environments/db-benchmarking/`

The outcome expected is something like:

```
Plan: 67 to add, 0 to change, 0 to destroy.

  Warning: Resource targeting is in effect

  You are creating a plan with the -target option, which means that the result of this plan

  The -target option is not for routine use, and is provided only for exceptional situations


Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value:
```

You should type yes if interested to create the environment.

Afterward you should have the following message in case of no errors:

```
module.pg12ute-patroni-source.google_compute_backend_service.default[0]: Still creating... [
module.pg12ute-patroni-source.google_compute_backend_service.default[0]: Creation complete a

Apply complete! Resources: 67 added, 0 changed, 0 destroyed.
```

How to reset the environment Terraform

Basically, you need to execute a tf destroy and tf apply from some modules.

Execute the tf destroy command in the folder from the benchmark environment:

folder: `environments/db-benchmarking/`

command: `tf destroy --target=module.pg12ute-patroni-source --target=module.pg12ute-patroni-target --target=module.ci-pgbouncer --target=module.ci-pgbouncer-sidekiq`

The output will be :

```
Plan: 0 to add, 0 to change, 67 to destroy.

  Warning: Resource targeting is in effect

  You are creating a plan with the -target option, which means that the result of this plan

  The -target option is not for routine use, and is provided only for exceptional situations


Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.
```

Type yes if you agree on destroying the environment.

The confirmation message should be similar to:

```
module.pg12ute-patroni-source.google_compute_subnetwork.subnetwork[0]: Destroying... [id=pro
module.pg12ute-patroni-source.google_compute_subnetwork.subnetwork[0]: Still destroying... [
module.pg12ute-patroni-source.google_compute_subnetwork.subnetwork[0]: Destruction complete

Destroy complete! Resources: 67 destroyed.
```

After you should execute a `tf apply` in the folder from the benchmark environment:

folder: `environments/db-benchmarking/`

command: `tf apply --target=module.pg12ute-patroni-source --target=module.pg12ute-patroni-ta`
`--target=module.ci-pgbouncer --target=module.ci-pgbouncer-sidekiq`

Output:

```
Plan: 67 to add, 0 to change, 0 to destroy.

  Warning: Resource targeting is in effect

  You are creating a plan with the -target option, which means that the result of this plan

  The -target option is not for routine use, and is provided only for exceptional situations


Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value:
```

Type yes to create the environment.

The confirmation message should be similar to:

```
module.pg12ute-patroni-source.google_compute_backend_service.default[0]: Still creating... [
module.pg12ute-patroni-source.google_compute_backend_service.default[0]: Creation complete a

Apply complete! Resources: 67 added, 0 changed, 0 destroyed.
```

*** It can take more than 15 minutes to provision the disks with the snapshot from production.

Alternatively, you can use replace/target Terraform features to only redeploy a specific set of instances for the source/target cluster

Recreating Source Cluster Nodes:

```
tf plan -replace="module.pg12ute-patroni-source.google_compute_instance.instance_with_attach
-replace="module.pg12ute-patroni-source.google_compute_instance.instance_with_attached_disk
-replace="module.pg12ute-patroni-source.google_compute_instance.instance_with_attached_disk
-replace="module.pg12ute-patroni-source.google_compute_disk.data_disk[0]" \
-replace="module.pg12ute-patroni-source.google_compute_disk.data_disk[1]" \
-replace="module.pg12ute-patroni-source.google_compute_disk.data_disk[2]" \
-target="module.pg12ute-patroni-source" \
-out=source-replace.plan
```

Recreating Target Cluster Nodes:

```
tf plan -replace="module.pg12ute-patroni-target.google_compute_instance.instance_with_attach
-replace="module.pg12ute-patroni-target.google_compute_instance.instance_with_attached_disk
-replace="module.pg12ute-patroni-target.google_compute_instance.instance_with_attached_disk
-replace="module.pg12ute-patroni-target.google_compute_disk.data_disk[0]" \
-replace="module.pg12ute-patroni-target.google_compute_disk.data_disk[1]" \
-replace="module.pg12ute-patroni-target.google_compute_disk.data_disk[2]" \
-target="module.pg12ute-patroni-target" \
-out=target-replace.plan
```

How to configure the Patroni clusters and the environment

1 - Stop the Patroni service in all the nodes with the command: `systemctl stop patroni`

2 - Remove the DCS entry from the Source cluster, executing : `gitlab-patronictl remove pg12ute-patroni-source`

Answer the name of the cluster that will be removed: pg12ute-patroni-source

Answer the confirmation message: `Yes I am aware`

3 - Remove the DCS entry from the Source target: `gitlab-patronictl remove pg12ute-patroni-target`

Answer the name of the cluster that will be removed: pg12ute-patroni-target

Answer the confirmation message: `Yes I am aware`

4 - Change the ownership from the data and log folders in all the hosts from both clusters(source and target):

```
cd /var/opt/gitlab/
chown -R gitlab-psql:gitlab-psql postgresql/
chown -R gitlab-psql:gitlab-psql patroni/
cd /var/log/gitlab/
chown -R gitlab-psql:gitlab-psql postgresql/
chown -R syslog:syslog patroni/
```

5 - delete the .dynamic.json file in all the target nodes:

```
rm -rf /var/opt/gitlab/postgresql/data12/patroni.dynamic.json
```

6 - Start the Patroni service on the source primary: `systemctl start patroni`. We execute the next steps in order since we want to ensure the source-01 is the primary from the source cluster—the target-01 as the Cascade Leader. The main reason is due to a configuration in the ansible-playbook for the switchover.

7 - check the status: `gitlab-patronictl list`

Output:

```
root@pg12ute-patroni-source-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal:/var/log
+ Cluster: pg12ute-patroni-source (6959847276950353765) ------------------------+----------
| Member                                                                        | Host
+-------------------------------------------------------------------------------+----------
| pg12ute-patroni-source-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal | 10.255.8.
+-------------------------------------------------------------------------------+----------
```

8 - Start the Patroni service on the source secondaries: `systemctl start patroni` after 30 seconds from starting the source-01. You do not need to wait for source 01 to be started completely.

9 - check the status: `gitlab-patronictl list`

Output:

```
root@pg12ute-patroni-source-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal:/var/log
+ Cluster: pg12ute-patroni-source (6959847276950353765) ------------------------+----------
| Member                                                                        | Host
+-------------------------------------------------------------------------------+----------
| pg12ute-patroni-source-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal | 10.255.8.
| pg12ute-patroni-source-02-db-db-benchmarking.c.gitlab-db-benchmarking.internal | 10.255.8.
| pg12ute-patroni-source-03-db-db-benchmarking.c.gitlab-db-benchmarking.internal | 10.255.8.

+-------------------------------------------------------------------------------+----------
```

10 - Check the leader assumed: `gitlab-patronictl list`

```
root@pg12ute-patroni-source-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal:/var/log
+ Cluster: pg12ute-patroni-source (6959847276950353765) ------------------------+----------
| Member                                                                        | Host
+-------------------------------------------------------------------------------+----------
| pg12ute-patroni-source-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal | 10.255.8.
| pg12ute-patroni-source-02-db-db-benchmarking.c.gitlab-db-benchmarking.internal | 10.255.8.
| pg12ute-patroni-source-03-db-db-benchmarking.c.gitlab-db-benchmarking.internal | 10.255.8.
+-------------------------------------------------------------------------------+----------
```

10 - Check the source cluster is running: `gitlab-patronictl list` .

```
root@pg12ute-patroni-source-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal:/var/log
+ Cluster: pg12ute-patroni-source (6959847276950353765) ------------------------+----------
| Member                                                                        | Host
+-------------------------------------------------------------------------------+----------
```

```
| pg12ute-patroni-source-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal | 10.255.8.
| pg12ute-patroni-source-02-db-db-benchmarking.c.gitlab-db-benchmarking.internal | 10.255.8.
| pg12ute-patroni-source-03-db-db-benchmarking.c.gitlab-db-benchmarking.internal | 10.255.8.
+--------------------------------------------------------------------------------+----------
```

11 - Start the Patroni service on the target primary: `systemctl start patroni`.
You can start this host 30 seconds after the secondaries from the source cluster.
You do not need to wait for the secondaries 01 to be started completely.

12 - Check the leader assumed: `gitlab-patronictl list`

Output:

```
root@pg12ute-patroni-target-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal:/var/log
+ Cluster: pg12ute-patroni-target (6959847276950353765) -------------------------+----------
| Member                                                                         | Host
+--------------------------------------------------------------------------------+----------
| pg12ute-patroni-target-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal | 10.255.9.
+--------------------------------------------------------------------------------+----------
```

13 - Start the Patroni service on the target secondaries: `systemctl start patroni`. You can start this host 30 seconds after the primary from the target cluster. You do not need to wait for the target 01 to be started completely.

14 - Check the leader assumed: `gitlab-patronictl list`

Output:

```
root@pg12ute-patroni-target-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal:/var/log
+ Cluster: pg12ute-patroni-target (6959847276950353765) -------------------------+----------
| Member                                                                         | Host
+--------------------------------------------------------------------------------+----------
| pg12ute-patroni-target-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal | 10.255.9.
| pg12ute-patroni-target-02-db-db-benchmarking.c.gitlab-db-benchmarking.internal | 10.255.9.
| pg12ute-patroni-target-03-db-db-benchmarking.c.gitlab-db-benchmarking.internal | 10.255.9.
+--------------------------------------------------------------------------------+----------
```

15 - wait until the target cluster is running : `gitlab-patronictl list`

Output:

```
root@pg12ute-patroni-target-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal:/var/log
+ Cluster: pg12ute-patroni-target (6959847276950353765) -------------------------+----------
| Member                                                                         | Host
+--------------------------------------------------------------------------------+----------
| pg12ute-patroni-target-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal | 10.255.9.
| pg12ute-patroni-target-02-db-db-benchmarking.c.gitlab-db-benchmarking.internal | 10.255.9.
| pg12ute-patroni-target-03-db-db-benchmarking.c.gitlab-db-benchmarking.internal | 10.255.9.
+--------------------------------------------------------------------------------+----------
```

How to setup other components from the environment from the environment

1 - At the jmeter boxes:

checkout the repo: `git clone git@gitlab.com:gitlab-com/gl-infra/db-migration.git`

here we will use the content on the folder: `benchmark`

*** If you need to release disk space, delete the last tests. The files are 'last-result.csv', and the content of the folder 'results'

2 - At the console box:

checkout the repo: `git clone git@gitlab.com:gitlab-com/gl-infra/db-migration.git`

here we will use the content on the folder: `db-sharding`

How to setup Netdata

Netdata is a light monitoring tool that we install on the hosts. For more info: https://www.netdata.cloud/

If you execute the setup from Netdata manually (on the primary from the source and target), we need to :

Stop chef-client: `chef-client-disable`

Output:

```
root@pg12ute-patroni-source-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal:/var/log
2021-09-02 13:33:31 UTC  LOG  (jfinotto)  Disabling chef-client.  Comment: No comment.
2021-09-02 13:33:31 UTC  LOG  (jfinotto)  Stopping and disabling chef-client systemd unit.
Removed symlink /etc/systemd/system/multi-user.target.wants/chef-client.service.
2021-09-02 13:33:31 UTC  LOG  (jfinotto)  Disabling chef-client executable.
2021-09-02 13:33:31 UTC  LOG  (jfinotto)  Replacing chef-client symlink.  The original will
2021-09-02 13:33:31 UTC  LOG  (jfinotto)  Updating prometheus metric "chef_client_enabled" w
2021-09-02 13:33:31 UTC  LOG  (jfinotto)  Successfully disabled chef-client.  Comment: No co
```

verify the status with: `systemctl status chef-client`

Output:

```
root@pg12ute-patroni-source-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal:/var/log
 chef-client.service - Chef Client daemon
   Loaded: loaded (/etc/systemd/system/chef-client.service; disabled; vendor preset: enabled
   Active: inactive (dead)
Sep 02 13:33:31 pg12ute-patroni-source-01-db-db-benchmarking chef-client[2459]: Chef Client
Sep 02 13:33:31 pg12ute-patroni-source-01-db-db-benchmarking chef-client[2459]: [2021-09-02T
Sep 02 13:33:31 pg12ute-patroni-source-01-db-db-benchmarking chef-client[2459]: [2021-09-02T
Sep 02 13:33:31 pg12ute-patroni-source-01-db-db-benchmarking chef-client[2459]: [2021-09-02T
Sep 02 13:33:31 pg12ute-patroni-source-01-db-db-benchmarking chef-client[2459]: ---- Begin c
```

```
Sep 02 13:33:31 pg12ute-patroni-source-01-db-db-benchmarking chef-client[2459]: STDOUT:
Sep 02 13:33:31 pg12ute-patroni-source-01-db-db-benchmarking chef-client[2459]: STDERR:
Sep 02 13:33:31 pg12ute-patroni-source-01-db-db-benchmarking chef-client[2459]: ---- End out
Sep 02 13:33:31 pg12ute-patroni-source-01-db-db-benchmarking chef-client[2459]: Ran ua statu
Sep 02 13:33:31 pg12ute-patroni-source-01-db-db-benchmarking systemd[1]: Stopped Chef Client
```

Edit the file net data config file : `/opt/netdata/etc/netdata/python.d/postgres.conf`

With the following content:

```
# https://github.com/netdata/netdata/blob/master/collectors/python.d.plugin/postgres/postgre

tcp:
  name: 'default'
  user: $DB_USER
  password: $DB_PWD
  database: 'gitlabhq_production'
  host: 'localhost'
  port: 5432
```

Restart Netdata in each host that has been configured: `systemctl stop netdata`

Execute locally, the redirect of the Netdata dashboard port (19999) via SSH to the local port 20000:

```
ssh -i ~/.ssh/id_rsa -L 20000:pg12ute-patroni-source-01-db-db-benchmarking.c.gitlab-db-bench
```

Execute locally, the redirect of the Netdata dashboard port (19999) via SSH to the local port 20001:

```
ssh -i ~/.ssh/id_rsa -L 20001:pg12ute-patroni-target-01-db-db-benchmarking.c.gitlab-db-bench
```

Locally in the browser, how to access Netdata source Postgresql data:

```
http://127.0.0.1:20000/#menu_postgres_default;after=-480;before=0;theme=slate
```

Locally in the browser, the URL to access Netdata target data:

```
http://127.0.0.1:20001/#;after=-480;before=0;theme=slate
```

How to setup PGWatch 2.0

PGWatch 2.0 is a detailed PostgreSQL monitoring that we are benchmarking. For more information: https://github.com/cybertec-postgresql/pgwatch2

Execute on the primary from the source cluster the following sql, connecting with `gitlab-psql`:

```
create role pgwatch2 login password $pgwatch2_pwd;
grant pg_monitor TO pgwatch2;
grant select on table pg_stat_replication to pgwatch2;
grant select on table pg_stat_activity to pgwatch2;
```

```
grant execute on function pg_stat_file(text) to pgwatch2;
grant execute on function pg_ls_dir(text) to pgwatch2;
grant connect on database gitlabhq_production to pgwatch2;
grant usage on schema public to pgwatch2;
if pg_wait_sampling extension is used:
GRANT EXECUTE ON FUNCTION pg_wait_sampling_reset_profile() TO pgwatch2;
```

Execute locally, the redirect of the Netdata dashboard port (19999) via SSH to the local port 3000:

```
ssh -fNTML 3000:localhost:3000 pgwatch-01-sv-db-benchmarking.c.gitlab-db-benchmarking.inte
```

The URL to access in the browser:

`http://127.0.0.1:3000/`

Reset database statistics

On the source primary execute, connecting with `gitlab-psql`:

```
select pg_stat_reset(), pg_stat_statements_reset()/*, pg_stat_kcache_reset()*/, pg_stat_rese
drop table if exists _benchmark_lsn;
create table _benchmark_lsn as select now() as created_at, pg_current_wal_lsn() as lsn;
```

On the target primary execute, connecting with `gitlab-psql`:

```
select pg_stat_reset(), pg_stat_statements_reset()/*, pg_stat_kcache_reset()*/, pg_stat_rese
```

Verify the number of connections on each primary database on the clusters source and target

Create a session on the primary from the source cluster:

Execute: `watch -n 1 "gitlab-psql -c \"select count(*) from pg_stat_activity where state!='IDLE';\""`

Create a session on the primary from the target cluster:

Execute: `watch -n 1 "gitlab-psql -c \"select count(*) from pg_stat_activity where state!='IDLE';\""`

How to start simulating the traffic to the database

In this step, we will start the traffic on the JMeter nodes, which will create database traffic to the pgbouncer nodes. The Pgbouncer nodes will redirect the traffic to the primary at source cluster.

JMeter01:

Path: `/db-migration/benchmark/bin/`

Session 01:

```
./run-bench.sh -h ci-pgbouncer.service.consul -d gitlabhq_production -U gitlab-superuser -p
```

Session 02:

```
./run-bench.sh -h ci-pgbouncer-sidekiq.service.consul -d gitlabhq_production -U gitlab-super
```

In the JMeter 02 instance, start:

Path: `/db-migration/benchmark/bin/`

Session 01:

```
./run-bench.sh -h pgbouncer.service.consul -d gitlabhq_production -U gitlab-superuser -p 643
```

Session 02:

```
./run-bench.sh -h pgbouncer-sidekiq.service.consul -d gitlabhq_production -U gitlab-superuse
```

How to start simulating the traffic to the patroni read replicas:

To generate traffic on source read replica:

```
./run-bench-secondaries.sh -h pg12ute-patroni-source-replica.service.consul -d gitlabhq_proc
```

To generate traffic on target read replica:

```
./run-bench-secondaries.sh -h pg12ute-patroni-target-replica.service.consul -d gitlabhq_proc
```

Additional test plans to run on read replica are available under `db-migration/benchmark/plans-secondaries` directory

How to execute the switchover or the activity that will be tested

Until now, we have done the setup for the environment, and we have all the monitoring ready to collect data and execute the change we want to execute in the benchmark environment.

In this test, we are executing the switchover and promotion of a new database cluster. After this ansible-playbook, we expect to split the traffic related to CI to the target database cluster:

Path: `/db-migration/db-sharding/`

Command: `export ENVIRONMENT=db-benchmarking; ansible-playbook -i inventory/db-benchmarking.yml playbooks/upgrade_to_sharding.yml`

Resumed output:

```
Thursday 02 September 2021  14:05:57 +0000 (0:00:01.174)       0:00:39.225 ****
===============================================================================
Gathering Facts ---------------------------------------------------------------
[Execution (5/8)] Ensure the newly promoted Primary Target database is discoverable by DNS -
[Intermediate-check (3/4)] Check if Primary Target LSN caught up with Primary Source LSN ---
[Execution (4/8)] Promote database on replica ---------------------------------
```

```
[Execution (3/8)] Update database config in the secondary pgbouncer cluster ---------------
[Execution (7/8)] Ensure the Primary Target is not in recovery mode ------------------------
[Execution (1/8)] Stop all traffic from the secondary pgbouncer cluster --------------------
[Execution (2/8)] Stop Chef client to prevent unexpected changes during the process --------
[Intermediate-check (2/4)] Get LSN from the Primary Target ---------------------------------
[Execution (6/8)] Check if the Primary Target database is ready ----------------------------
[Intermediate-check (1/4)] Get LSN from the Primary Source ---------------------------------
[Pre-check (1/2)] Check PostgreSQL streaming replication lag is lower than 5MB -------------
[Pre-check (2/2)] Check if PostgreSQL streaming replication lag is smaller than 5MB before p
[Intermediate-check (4/4)] Show Source and Target Primary LSN ------------------------------
[Execution (8/8)] Restore all traffic on the secondary pgbouncer cluster -------------------
```

How to collect metrics and reports

After the switchover, please verify the number of connections on both sides.

After few minutes, stop the JMeter sessions.

Export data from Netdata from source and target

Now In the Netdata GUI on the HTTP address: `http://127.0.0.1:20000/`

Export the data and attach the compressed file in the issue where you are tracking your activity.

Collect PostgreSQL from the switchover exercise

We should follow those steps in the primary from source and target.

Create the following script perf.sh:

```
#!/bin/sh
function collect_results() {
  local run_number=$1
  let run_number=run_number+1
## let PSQL_BINARY="/usr/local/bin/gitlab-psql"

  ## Get statistics
  OP_START_TIME=$(date +%s)

  out=$(/usr/local/bin/gitlab-psql -c "select sum(total_time) from pg_stat_statements where
  PG_STAT_TOTAL_TIME=${out//[!0-9.]/}

  for table2export in \
    "pg_settings order by name" \
    "pg_stat_statements order by total_time desc" \
    "pg_stat_archiver" \
    "pg_stat_bgwriter" \
```

```
    "pg_stat_database order by datname" \
    "pg_stat_database_conflicts order by datname" \
    "pg_stat_all_tables order by schemaname, relname" \
    "pg_stat_xact_all_tables order by schemaname, relname" \
    "pg_stat_all_indexes order by schemaname, relname, indexrelname" \
    "pg_statio_all_tables order by schemaname, relname" \
    "pg_statio_all_indexes order by schemaname, relname, indexrelname" \
    "pg_statio_all_sequences order by schemaname, relname" \
    "pg_stat_user_functions order by schemaname, funcname" \
    "pg_stat_xact_user_functions order by schemaname, funcname" \
  ; do
    /usr/local/bin/gitlab-psql -b -c "copy (select * from $table2export) to stdout with csv
  done

  /usr/local/bin/gitlab-psql -b -c "
    copy (
      select
        pg_current_wal_lsn() - lsn as wal_bytes_generated,
        pg_size_pretty(pg_current_wal_lsn() - lsn) wal_pretty_generated,
        pg_size_pretty(3600 * round(((pg_current_wal_lsn() - lsn) / extract(epoch from now()
      from _benchmark_lsn
    ) to stdout with csv header delimiter ',';" > wal_stats.${run_number}.csv
}

collect_results
```

Add the rights to execute on the file.

```
sudo chmod +x perf.sh
```

Execute the script:

```
bash perf.sh
```

The output will be similar to:

```
ls -lha *.csv
rw-r--r--  1 root     root      66K Aug 26 06:30 pg_settings.1.csv
-rw-r--r--  1 root     root     286K Aug 26 06:30 pg_stat_all_indexes.1.csv
-rw-r--r--  1 root     root     117K Aug 26 06:30 pg_stat_all_tables.1.csv
-rw-r--r--  1 root     root      148 Aug 26 06:30 pg_stat_archiver.1.csv
-rw-r--r--  1 root     root      275 Aug 26 06:30 pg_stat_bgwriter.1.csv
-rw-r--r--  1 root     root      713 Aug 26 06:30 pg_stat_database.1.csv
-rw-r--r--  1 root     root      201 Aug 26 06:30 pg_stat_database_conflicts.1.csv
-rw-r--r--  1 root     root     280K Aug 26 06:30 pg_statio_all_indexes.1.csv
-rw-r--r--  1 root     root      23K Aug 26 06:30 pg_statio_all_sequences.1.csv
-rw-r--r--  1 root     root      74K Aug 26 06:30 pg_statio_all_tables.1.csv
-rw-r--r--  1 root     root     168K Aug 26 06:30 pg_stat_statements.1.csv
-rw-r--r--  1 root     root       54 Aug 26 06:30 pg_stat_user_functions.1.csv
```

```
-rw-r--r--  1 root     root      75K Aug 26 06:30 pg_stat_xact_all_tables.1.csv
-rw-r--r--  1 root     root       54 Aug 26 06:30 pg_stat_xact_user_functions.1.csv
-rw-r--r--  1 root     root       82 Aug 26 06:30 wal_stats.1.csv
```

Tar all the csv files:

```
tar -czf source01.tar.gz *.csv
```

Scp locally the file and attach to your issue for further analysis.

Additionally, you could execute this query for analysis in both primary hosts for comparison, connecting with `gitlab-psql`:

```
select substr(query , 1,40 ) , queryid , calls , total_time , min_time , max_time, mean_time
```

And the output is:

```
                substr                  |       queryid        | calls  |      total_time
----------------------------------------+----------------------+--------+----------------
 SELECT "ci_pipelines".* FROM "ci_pipelin |  6673794014151945635 | 451401 |    23601.000355
 SELECT "ci_builds".* FROM "ci_builds" WH | -2106760600543814756 | 325081 |    34697.345718
 SELECT "ci_builds".* FROM "ci_builds" WH |   388151056235919956 | 304006 |    25532.359276
 SELECT "ci_job_artifacts".* FROM "ci_job | -4022738317710124596 | 224861 |   16533.755325
 SELECT "ci_runners".* FROM "ci_runners"  | -1216497675517520397 | 221929 |    8776.029135
 SELECT "ci_build_trace_chunks".* FROM "c |   -52026140613848565 | 198474 |   1975.073745
 SELECT "ci_runners".* FROM "ci_runners"  | -8879233226840007005 | 164150 |     9351.84250
 SELECT "ci_builds".* FROM "ci_builds" WH |  4267872043094895864 | 155115 |    202344.1687
 SELECT $1 AS one FROM "ci_builds"       +|  1599479802630180052 | 152011 |     79166.0017
    WH                                   |                      |        |
 SELECT "ci_build_trace_chunks".* FROM "c |   958142551783346735 | 123941 |   1309.095388
 SELECT "ci_stages".* FROM "ci_stages" WH |  6699769169063417733 | 108883 |    13064.12745
 SELECT MAX(id), "ci_builds"."name" FROM  | -8560280642564996600 |  92131 |    41031.23721
 SELECT pg_catalog.to_char(pg_catalog.pg_ |  7911798196226659224 |   4717 |    628.4049570
 select count(*) from pg_stat_activity wh | -9104946687408020398 |   1761 |    724.0960050
 SELECT                                  +|  5921977009657797056 |   1406 |    352.3714829
     application_name,                   +|                      |        |
     pg_wal_                              |                      |        |
 SELECT                                  +|   369209257336299428 |   1406 |    976.1307710
```

Processing the JMeter result files

First, we need to copy the jmeter result files locally.

Here is an example of one SCP:

```
scp jmeter-02-inf-db-benchmarking.c.gitlab-db-benchmarking.internal://home/user/db-migratio
```

Now in the GUI from JMeter, we need to load one of the workloads.

Inside we need to add a component named: `Aggregated Report`

The menu path is :

- 1 - Right-click on the `Thread Group`
- 2 - Click in `Add`
- 3 - Click in the submenu in the option `Listener`
- 4 - Click in the submenu in the option `Aggregated Report`

In the Aggregated Report, we need to load the result file. Click in the `Browse` button in front of the field `Filename`.

Depending on the volume of data can take some minutes to process the data.

After we have the data processed, we can click on the button `Save Table Data`.

The output will be similar to:

```
Label,# Samples,Average,Median,90% Line,95% Line,99% Line,Min,Max,Error %,Throughput,Receive
queryid -7715180042597491341 ,777512,1,1,2,2,4,0,1023,47.853%,2519.51419,197.98,0.00
queryid 6699769169063417733,426482,1,1,2,2,4,0,274,47.854%,1382.01650,362.12,0.00
queryid -8451338199510579657,880852,1,1,2,2,4,0,1013,47.853%,2854.38567,588.69,0.00
queryid 8345325569152878536,633164,0,1,2,2,6,0,1028,100.000%,2051.77694,143.24,0.00
queryid 6673794014151945635,1768269,1,1,2,3,5,0,1024,47.853%,5730.04511,2011.69,0.00
queryid 7760111794549822367 ,485534,0,1,2,2,4,0,289,47.853%,1573.36963,127.97,0.00
queryid 388151056235919956,1190872,1,1,3,3,5,0,1028,47.854%,3859.00012,5428.50,0.00
queryid 1599479802630180052 ,595435,1,1,3,3,6,0,1630,47.854%,1929.50307,132.44,0.00
queryid -2106760600543814756,1272891,1,1,3,3,6,0,1025,47.854%,4124.78127,6040.53,0.00
queryid -8560280642564996600 ,360870,1,1,3,3,6,0,8381,47.854%,1169.39299,42.06,0.00
queryid 4267872043094895864 ,606910,3,1,7,17,56,0,9352,47.855%,1966.69410,119400.22,0.00
queryid -1216497675517520397 ,869371,1,1,2,2,4,0,1028,47.853%,2817.18169,844.83,0.00
queryid -8879233226840007005 ,643006,1,1,2,2,4,0,1024,47.853%,2083.64982,625.84,0.00
TOTAL,10511168,1,1,2,3,7,0,9352,50.995%,34061.25809,135945.32,0.00
```

I recommend improving the visibility to everyone use any tool that converts from '*.csv' files to Markdown Language.

I used a web page like: https://www.convertcsv.com/csv-to-markdown.htm

and the output will be similar to:

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Max	Error %	Throughput	Received KB/sec	Sent KB/sec
queryid -7715180042597491341	777512	1	1	2	2	4	0	1023	47.853%	2519.51419	197.98	0.00
queryid 6699769169063417733	426482	1	1	2	2	4	0	274	47.854%	1382.01650	362.12	0.00
queryid -8451338199510579657	880852	1	1	2	2	4	0	1013	47.853%	2854.38567	588.69	0.00
queryid 8345325569152878536	633164	0	1	2	2	6	0	1028	100.000%	2051.77694	143.24	0.00

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Max	Error %	Throughput	Received KB/sec	Sent KB/sec
queryid 6673794014151945635	176826	9	1	2	3	5	0	1024	47.853%	30.04251	11.69	0.00
queryid 7760111794549822367	48553	0	1	2	2	4	0	289	47.853%	73.36126	7.97	0.00
queryid 3881510562359199 56	119087	2	1	3	3	5	0	1028	47.854%	59.00501	28.50	0.00
queryid 1599479802630180052	59543	1	1	3	3	6	0	1630	47.854%	29.50132	7.44	0.00
queryid - 2106760600543814756	127289	1	1	3	3	6	0	1025	47.854%	24.78612	7.53	0.00
queryid - 8560280642564996600	36087	1	1	3	3	6	0	838	47.854%	69.39229	9.06	0.00
queryid 4267872043094895864	60691	3	1	7	17	56	0	935	47.855%	66.69419	400.22	0.00
queryid - 1216497675517520397	86937	1	1	2	2	4	0	1028	47.853%	17.18846	4.83	0.00
queryid - 8879233226840007005	64300	1	1	2	2	4	0	1024	47.853%	26.83649	7.84	0.00
TOTAL	1051116	8	1	2	3	7	0	9352	50.995%	4061.25	1138.45	0.00

It is possible to generate different reports from JMeter. Another one I use sometimes is the `Summary Report`.